Convergence and Accuracy of Physics-Informed Neural Networks versus Classical Finite Difference Schemes for Boundary Value Problems

Evan Strasdin
V00907185
MATH 449 Scientific Computing
Spring 2026

April 15, 2026

Abstract

Physics-informed neural networks (PINNs) have been proposed as a mesh-free alternative to classical discretisation schemes for ordinary and partial differential equations. This report asks whether, on well-posed one-dimensional boundary value problems where classical finite difference methods (FDM) have complete convergence theory, a PINN is competitive with the classical alternative. We implement FDM and PINN solvers for three BVPs of increasing difficulty (linear constant-coefficient, linear variable-coefficient, and nonlinear) and compare them using the tools taught in the course: convergence rates, condition number analysis, and wall-clock timing. FDM exhibits the expected second-order convergence and reaches errors below 10^{-6} in microseconds. The PINN error plateaus at roughly 10^{-5} regardless of the number of collocation points used, at a wall-clock cost four to five orders of magnitude larger. Double-precision floating-point error is ruled out as the cause of the plateau via an auxiliary experiment. For the class of problems considered here the classical methods are strictly superior.

Contents

1 Introduction **2**

2 Test Problems **3**
2.1 Problem 1: Linear, Constant Coefficient . . . 3
2.2 Problem 2: Linear, Variable Coefficient . . . 3
2.3 Problem 3: Nonlinear . . . 3

3 Finite Difference Methods **4**
3.1 Discretisation . . . 4
3.2 Linear System Structure and Conditioning . . . 5
3.3 Convergence Theorem . . . 5
3.4 Nonlinear Solvers . . . 5
3.4.1 Shooting Methods . . . 6
3.4.2 Picard and Newton FD . . . 6
3.5 FDM Numerical Results . . . 7

4 Physics-Informed Neural Networks as Nonlinear Collocation **7**
4.1 Course Context: PINNs as Nonlinear Collocation . . . 7
4.2 Formulation . . . 8
4.3 Architecture and Training . . . 9
4.4 PINN Numerical Results . . . 10

5 Comparison **10**
5.1 Accuracy versus Sampling Density . . . 10
5.2 Conditioning versus Loss Landscape . . . 11
5.3 Iterative Solver Convergence Rates . . . 11
5.4 Computational Cost . . . 12

6 Conclusions **12**

7 Further Research **13**

1 Introduction

Finite difference methods (FDM) for boundary value problems (BVPs) are one of the oldest and best-understood topics in scientific computing. Given a discretisation spacing h, the second-order centered difference stencil produces a tridiagonal linear system whose solution is provably second-order accurate in h whenever the discrete operator is invertible with a bounded inverse [1, 4]. Condition number analysis, truncation error, and iterative nonlinear solvers are all covered by the course notes.

Physics-informed neural networks (PINNs), introduced by Raissi et al. [6], replace the discrete grid with a neural network $\hat{u}(x;\theta)$ whose weights θ are trained by minimising the mean squared PDE residual at a collection of collocation points. Because the network is differentiable and the residual is computed via automatic differentiation, there is no discretisation error in the derivatives themselves. The collocation points are not required to form a mesh, so the method is nominally mesh-free and extends to irregular geometries and high dimensions.

In contrast to FDM, the convergence of a PINN is governed by the trajectory of a non-convex optimiser rather than by a deterministic linear solve. Existing analyses of the method's failure modes [5, 9] suggest that PINNs struggle to reach the kind of absolute accuracy that classical methods reach routinely. The question of this project is therefore: *on problems where the classical method has complete theory, is the PINN competitive?*

We answer this question by implementing both methods on three test BVPs taken directly from the MATH 449 Chapter 4 slides [2], and measuring the same quantities on both sides. Specifically, we ask:

1. **Accuracy versus resolution.** How does the L^∞ error scale with the FDM grid size n and with the PINN collocation count N_c?

2. **Iterative convergence on the nonlinear problem.** How do the four course solvers (bisection shooting, Newton shooting, Picard FD, Newton FD) compare against each other, and how does PINN optimisation compare against them?

3. **Conditioning and failure modes.** How does the classical FDM failure mode (condition-number-limited roundoff) compare against the empirical PINN plateau?

4. **Computational cost.** What is the wall-clock cost of reaching a fixed accuracy with each method?

The remainder of the paper is organised as follows. Section 2 states the three test problems and justifies the choices. Section 3 describes the FDM discretisation, convergence theory, conditioning, and nonlinear solvers, and reports FDM numerical results. Section 4 frames PINNs as a nonlinear generalisation of classical collocation, gives the formulation and training recipe, and reports PINN numerical results. Section 5 puts the two methods side by side on accuracy, conditioning, convergence rates, and cost. Section 6 summarises the findings.

2 Test Problems

All three test problems are second-order BVPs on $[0, 1]$, chosen from the course materials so that their well-posedness and (for two of them) exact solutions are already established [2, 3]. A method's L^∞ error is measured against the exact solution where one is available, and against a high-accuracy reference solution for Problem 3.

2.1 Problem 1: Linear, Constant Coefficient

The first test problem is the canonical one-dimensional Poisson equation with homogeneous Dirichlet boundary conditions:

$$u''(x) = -\pi^2 \sin(\pi x), \quad u(0) = 0, \quad u(1) = 0, \tag{1}$$

with exact solution $u_e(x) = \sin(\pi x)$. Problem 1 yields the canonical symmetric tridiagonal stiffness matrix whose eigenvalues and condition number are known analytically [3, §5.4], which makes it the cleanest benchmark for the conditioning study in section 5.2.

The choice of a non-polynomial exact solution is deliberate. A polynomial solution would be exactly representable by a neural network up to machine precision, so a polynomial test problem would hide any accuracy floor associated with the PINN optimiser. The sinusoidal exact solution exercises the spectral-bias failure mode that Rahaman et al. [5] and Wang et al. [8] identify as central to PINN training behaviour.

2.2 Problem 2: Linear, Variable Coefficient

The second problem has a variable coefficient and a non-homogeneous right-hand side, and is written in conservative form:

$$\left[(1 + x^2)u'(x)\right]' = d(x), \quad u(0) = 1, \quad u(1) = 2 + \sin(1), \tag{2}$$

where the forcing $d(x)$ is constructed so that the exact solution is

$$u_e(x) = x^2 + \sin(x) + 1.$$

Expanding the conservative form gives $d(x) = 2+6x^2+2x\cos(x)-(1+x^2)\sin(x)$, which we use to verify the discretisation. This problem is the natural test case for symmetric differencing of $(a(x)u')'$, covered in the Chapter 4 slides [2].

2.3 Problem 3: Nonlinear

The third problem is nonlinear and has no closed-form solution:

$$u''(x) = 3\,u(x) + x^2 + 10\,u(x)^3, \quad u(0) = 0, \quad u(1) = \beta, \tag{3}$$

with $\beta = 0.1$. The cubic term on the right-hand side is monotone increasing in u, which guarantees a unique classical solution for sufficiently small β. In the assignment treatment of this problem [2] a critical value

$\beta_c \approx 3.95$ is identified beyond which Newton shooting diverges; we pick $\beta = 0.1$ to stay safely below this value so that all of our nonlinear solvers are in their well-behaved regime.

Because there is no closed-form solution, we define a reference solution for error measurement: Newton shooting (section 3.4.1) run to a tolerance of 10^{-12} on the shooting residual, with the resulting IVP integrated by `scipy.integrate.solve_ivp` (RK45, tolerances at 10^{-12}). This reference is two to three orders of magnitude tighter than any error we expect from the tested methods, so the reference residual is negligible in our comparisons.

3 Finite Difference Methods

The classical side of the comparison is a standard FDM solver following the course notes [3]. We describe the discretisation, the tridiagonal system that results, the convergence and conditioning theorems that guarantee accuracy, and the four nonlinear solvers needed for Problem 3. FDM numerical results follow in section 3.5.

3.1 Discretisation

We place n interior grid points $x_i = ih$, $i = 1, \ldots, n$, on $[0, 1]$, with uniform spacing $h = 1/(n + 1)$. The boundary values $u_0 = u(0)$ and $u_{n+1} = u(1)$ are known from the boundary conditions; the unknowns are the vector $\mathbf{U} = (u_1, \ldots, u_n)^\top$ of approximations to u at the interior grid points.

Centered difference (Problems 1 and 3). For Problem 1 and for the linearised discretisations of Problem 3 we use the standard three-point centered second difference

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}. \tag{4}$$

Taylor expanding $u(x_i \pm h)$ to fourth order and adding the two expansions cancels the first and third derivative terms; after dividing through by h^2 the leading error term is $-\frac{h^2}{12}\, u^{(4)}(\xi_i)$, which is the origin of the $\mathcal{O}(h^2)$ truncation error.

Symmetric differencing for Problem 2. Applying the centered stencil directly to $(au')'$ with $a(x) = 1 + x^2$ is incorrect: the conservative form requires that the outer derivative be discretised as a difference of the inner flux au' at staggered half-points. Following [2] we use

$$\big[(au')'\big](x_i) \approx \frac{a_{i+1/2}\,(u_{i+1} - u_i) - a_{i-1/2}\,(u_i - u_{i-1})}{h^2}, \tag{5}$$

with $a_{i\pm1/2} = a\Big(\frac{x_i + x_{i\pm1}}{2}\Big)$. A Taylor expansion analogous to the constant-coefficient case shows that the leading error is again $\mathcal{O}(h^2)$, and that the resulting system is symmetric, which matters for the conditioning analysis below.

Boundary corrections. When u_0 and u_{n+1} are nonzero (as in Problem 2), the left-hand side of the stencil at $i = 1$ involves u_0 and the stencil at $i = n$ involves u_{n+1}. These known values are moved to the right-hand side as boundary correction terms u_0/h^2 and u_{n+1}/h^2 (with the appropriate $a_{i\pm1/2}$ prefactor for Problem 2). Omitting these corrections destroys the convergence rate on non-homogeneous problems.

3.2 Linear System Structure and Conditioning

Both discretisations produce a tridiagonal system $A_h \mathbf{U} = \mathbf{F}$, where $\mathbf{F}$ bundles the right-hand side forcing with the boundary correction terms. Tridiagonal systems are solved in $\mathcal{O}(n)$ arithmetic operations by the Thomas algorithm or an equivalent sparse LU factorisation, so each FDM solve is essentially free compared to a dense system.

For Problem 1, A_h is the standard symmetric, negative-definite Poisson matrix with eigenvalues

$$\lambda_k(A_h) = \frac{2\cos(k\pi h) - 2}{h^2}, \qquad k = 1, \ldots, n, \tag{6}$$

obtained by the characteristic-equation method [2]. The spectrum is negative, and the condition number of $|A_h|$ is

$$\kappa(A_h) = \frac{|\lambda_n|}{|\lambda_1|} \approx \frac{4(n+1)^2}{\pi^2} = \mathcal{O}(n^2), \tag{7}$$

where the approximation comes from expanding $\cos$ at both ends of the spectrum. This $\mathcal{O}(n^2)$ growth is the source of the FDM accuracy ceiling: any floating-point roundoff of relative size ϵ_{mach} in $\mathbf{F}$ is amplified by at most $\kappa(A_h)$ in the computed $\mathbf{U}$, so refinement eventually hits a wall where additional discretisation accuracy is overwhelmed by linear-algebra roundoff. For double precision $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$, this wall appears around $n \sim 10^7$, well beyond anything we use here.

3.3 Convergence Theorem

With the discretisation fixed, the key theoretical input is the BVP convergence theorem [2]. Let $\mathbf{u}^h$ be the restriction of the true solution to the grid, let $\mathbf{U}^h$ be the discrete solution, and define the error $\mathbf{E}^h = \mathbf{u}^h - \mathbf{U}^h$. Linearity gives

$$A_h \mathbf{E}^h = \boldsymbol{\tau}^h, \tag{8}$$

where $\boldsymbol{\tau}^h$ is the vector of truncation errors at the interior grid points. Taking norms,

$$\left\|\mathbf{E}^h\right\|_\infty \leq \left\|A_h^{-1}\right\|_\infty \left\|\boldsymbol{\tau}^h\right\|_\infty . \tag{9}$$

Since the truncation error is $\mathcal{O}(h^2)$ from eqs. (4) and (5) and $\left\|A_h^{-1}\right\|_\infty$ is bounded uniformly in h (provably so for the symmetric Poisson case and verified numerically for Problem 2), we obtain

$$\left\|\mathbf{E}^h\right\|_\infty = \mathcal{O}(h^2), \tag{10}$$

i.e. second-order convergence. We verify this empirically by computing the effective convergence rate between successive refinements,

$$r_h = \frac{\log\left(\left\|\mathbf{E}^h\right\|_\infty / \left\|\mathbf{E}^{h/2}\right\|_\infty\right)}{\log 2}, \tag{11}$$

and checking that $r_h \to 2$. Note that this is not the Lax equivalence theorem: Lax equivalence concerns stability and consistency for time-marching schemes, whereas the BVP convergence result is a straightforward consequence of invertibility and a bounded inverse.

3.4 Nonlinear Solvers

Problem 3 is nonlinear in the unknown u, so the simple linear solve of section 3.2 no longer applies. We implement and compare four iterative solvers, two based on shooting (which converts the BVP to a sequence of IVPs) and two on finite differences directly (which stay on the grid).

3.4.1 Shooting Methods

Rewriting eq. (3) as a first-order system with $y_1 = u$, $y_2 = u'$ gives

$$y_1' = y_2, \qquad y_2' = 3y_1 + x^2 + 10y_1^3, \qquad y_1(0) = 0, \quad y_2(0) = \alpha, \tag{12}$$

with α an unknown initial slope. The BVP is satisfied when $y_1(1;\alpha) = \beta$, i.e. when the residual function

$$\Phi(\alpha) = y_1(1;\alpha) - \beta \tag{13}$$

has a root.

Bisection shooting. If we can find an interval $[\alpha_\ell, \alpha_r]$ on which Φ changes sign, bisection is guaranteed to converge to a root, linearly, at a rate of one bit per iteration. It is essentially an oracle: at no point does it need derivative information. For Problem 3 at $\beta = 0.1$, initialised on $[\alpha_\ell, \alpha_r] = [-1, 1]$ and iterated until $|\Phi(\alpha)| < 10^{-12}$, bisection converges in 41 iterations.

Newton shooting. The Newton update requires $\Phi'(\alpha) = \partial y_1(1;\alpha)/\partial\alpha$. This is obtained by integrating the sensitivity equation alongside eq. (12). Let $z_1 = \partial y_1/\partial\alpha$, $z_2 = \partial y_2/\partial\alpha$; differentiating eq. (12) with respect to α gives

$$z_1' = z_2, \qquad z_2' = (3 + 30y_1^2)\, z_1, \qquad z_1(0) = 0, \quad z_2(0) = 1, \tag{14}$$

and $\Phi'(\alpha) = z_1(1)$. With a reasonable starting guess Newton converges quadratically; for Problem 3 at $\beta = 0.1$, starting from $\alpha_0 = 0$ and iterating to the same 10^{-12} tolerance, Newton shooting converges in **5** iterations, compared to 41 for bisection.

3.4.2 Picard and Newton FD

The direct alternative is to discretise the nonlinear BVP on the grid as $F(\mathbf{V}) = A_h\mathbf{V} - H(\mathbf{V}) = \mathbf{0}$, where $H(\mathbf{V})$ is the nonlinearity evaluated pointwise and A_h is the centered-difference matrix.

Picard FD (fixed-point). At each step solve the linear system

$$A_h\mathbf{V}^{k+1} = H(\mathbf{V}^k), \tag{15}$$

i.e. freeze the nonlinearity at the previous iterate. Each iteration is a single tridiagonal solve; convergence is linear when the fixed-point map is a contraction but is not guaranteed in general. For Problem 3 at $n = 127$ with $\mathbf{V}^0 = \mathbf{0}$ the iteration converges in **13** steps to a residual norm below 10^{-12}.

Newton FD. A Newton step on the full nonlinear system,

$$DF(\mathbf{V}^k)\,\Delta\mathbf{V}^k = -F(\mathbf{V}^k), \quad \mathbf{V}^{k+1} = \mathbf{V}^k + \Delta\mathbf{V}^k, \tag{16}$$

requires the Jacobian $DF(\mathbf{V}^k)$. For Problem 3 this is the tridiagonal matrix A_h plus a diagonal update $-(3 + 30(V_i^k)^2)$, so the Jacobian remains tridiagonal and each step is still an $\mathcal{O}(n)$ tridiagonal solve. With the zero initial guess sufficiently close to the solution at $\beta = 0.1$ the method converges quadratically in **2** iterations.

Both FD iterations have the same per-iteration cost as bisection or Newton shooting, but they stay on the grid, so their output is directly comparable with the linear FDM results and does not require an IVP solver.

3.5 FDM Numerical Results

Table 1 reports the L^∞ error of the FDM solution on the three problems as n doubles from 7 to 1023, together with the observed convergence rate from eq. (11). For all three problems the rate approaches 2 cleanly and the error is reduced by roughly a factor of four at each refinement, exactly as predicted by the BVP convergence theorem. That this holds for Problem 3 as well is worth emphasising: the nonlinear solver (Newton FD) is run to convergence at each grid, so the residual of the nonlinear system is negligible and the error is dominated by the truncation term.

	Problem 1		**Problem 2**		**Problem 3**	
n	L^∞ error	r_h	L^∞ error	r_h	L^∞ error	r_h
7	1.26×10^{-2}	–	3.91×10^{-3}	–	3.75×10^{-4}	–
15	3.14×10^{-3}	2.00	9.81×10^{-4}	2.00	9.54×10^{-5}	1.97
31	7.79×10^{-4}	2.01	2.45×10^{-4}	2.00	2.39×10^{-5}	2.00
63	1.92×10^{-4}	2.02	6.14×10^{-5}	2.00	5.97×10^{-6}	2.00
127	4.86×10^{-5}	1.98	1.53×10^{-5}	2.00	1.49×10^{-6}	2.00
255	1.18×10^{-5}	2.04	3.84×10^{-6}	2.00	3.73×10^{-7}	2.00
511	3.07×10^{-6}	1.94	9.59×10^{-7}	2.00	9.33×10^{-8}	2.00
1023	7.55×10^{-7}	2.02	2.40×10^{-7}	2.00	—	—

Table 1: FDM convergence on the three test problems. The observed rate r_h is computed between successive refinements via eq. (11); all three problems converge at the predicted second-order rate.

Figure 1 compares the four nonlinear solvers on Problem 3 at $n = 127$. Bisection shows the expected linear envelope and takes 41 iterations to reach the 10^{-12} tolerance; Newton shooting reaches the same tolerance in 5. Picard FD is linear and requires 13 iterations; Newton FD reaches the tolerance in 2. The Newton methods demonstrate the quadratic convergence guaranteed when the Jacobian is nonsingular and the initial guess is close enough to the solution. Picard and bisection are on the same linear-convergence tier but with different prefactors: Picard's linear rate is set by the contraction constant of the fixed-point map, while bisection's is fixed at exactly $1/2$ per iteration.

4 Physics-Informed Neural Networks as Nonlinear Collocation

PINNs are framed below as a nonlinear generalisation of classical collocation, followed by the formulation, the hard-boundary-condition ansatz, and the training recipe.

4.1 Course Context: PINNs as Nonlinear Collocation

The Chapter 4 slides cover three projection-based approaches to BVPs: *collocation* [2], *Galerkin*, and *B-spline / FEM*. In classical linear collocation one picks a fixed basis $\{\phi_k(x)\}_{k=1}^M$, writes the approximate solution as a linear combination

$$\hat{u}(x) = \sum_{k=1}^{M} c_k \phi_k(x), \tag{17}$$

and enforces the PDE residual to be zero at N_c collocation points $\{x_j\}_{j=1}^{N_c}$. This produces a linear system in the coefficients c_k which can be solved directly.

PINNs are exactly this construction with the linear span of a fixed basis replaced by the nonlinear function class represented by a neural network. Table 2 summarises the correspondence.

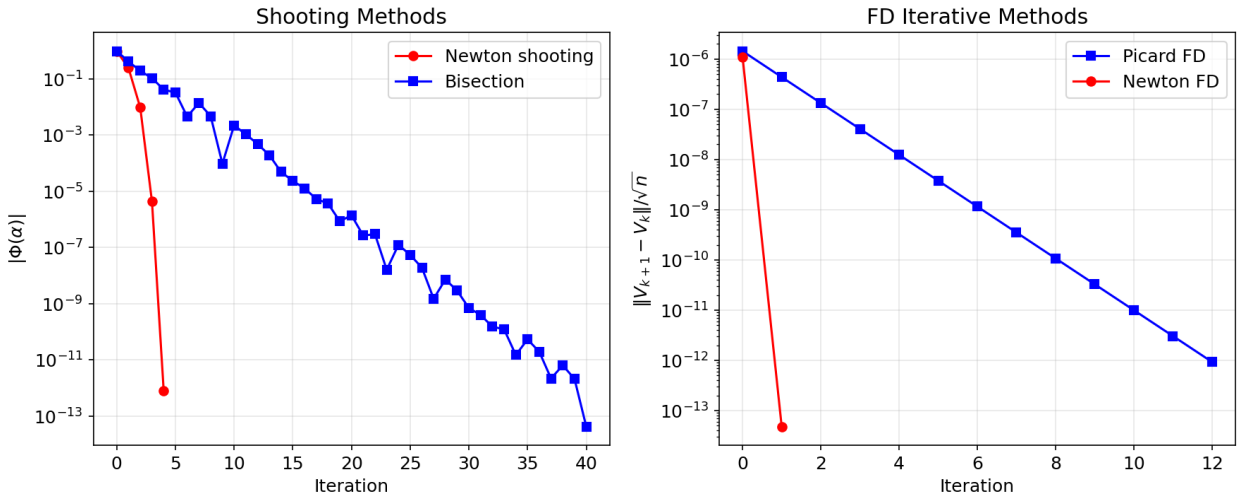

Figure 1: Convergence histories of the four nonlinear solvers on Problem 3 ($\beta = 0.1$, $n = 127$, tolerance 10^{-12}). Left: shooting methods on $|\Phi(\alpha)|$, where Newton converges quadratically in 5 iterations and bisection linearly in 41. Right: FD iterations on the discrete residual, where Newton FD reaches the tolerance in 2 steps and Picard FD in 13.

	Linear collocation	**PINN**
Approximant	$\sum c_k \phi_k(x)$	neural network $\hat{u}(x;\theta)$
Unknowns	linear coefficients c_k	network weights θ
Residual	at N_c collocation points	at N_c collocation points
Solve method	one linear system	nonlinear optimisation
Mesh requirement	structured basis	any scattered points
Convergence	provable (from basis theory)	no guarantee

Table 2: Correspondence between classical collocation and PINNs.

Degrees of freedom. One distinction that drives the comparison plots in section 5.1 is that the parameter count and the sampling density are separate knobs in a PINN, whereas in FDM they are the same quantity. For FDM, n is simultaneously the grid size, the number of unknowns, and the sampling density, and refining the grid improves both resolution and representational capacity. For a PINN the number of trainable weights is fixed by the architecture (roughly 3200 for ours), and N_c is the number of points at which the PDE residual is evaluated. One can increase N_c without changing the network, or enlarge the network without adding collocation points. Sweeping N_c therefore answers the question *does more sampling help a fixed model?*, which is a genuinely different question from the one FDM refinement answers.

4.2 Formulation

The PINN approximates the solution of the BVP by a neural network $\mathrm{NN}(x;\theta)$ with trainable parameters θ. To ensure that the boundary conditions are satisfied exactly at every training step we use the hard-boundary-condition ansatz

$$\hat{u}(x;\theta) = g_0(1-x) + g_1 x + x(1-x)\,\mathrm{NN}(x;\theta), \tag{18}$$

where $g_0 = u(0)$, $g_1 = u(1)$. The first two terms are the straight line from g_0 to g_1 and already satisfy both boundary conditions; the factor $x(1-x)$ in the third term vanishes at both endpoints, so the network's

contribution is zero on the boundary regardless of θ. An alternative is to add a soft penalty term to the loss for boundary violations, but hard enforcement gives strictly better numerical behaviour and removes a tunable weight from the loss.

For each problem the loss is the mean squared residual of the governing equation at N_c uniformly spaced interior collocation points $\{x_j\}_{j=1}^{N_c}$:

$$\mathcal{L}(\theta) = \frac{1}{N_c} \sum_{j=1}^{N_c} \left| \mathcal{R}[\hat{u}](x_j; \theta) \right|^2, \tag{19}$$

where $\mathcal{R}$ is the differential operator rearranged to have zero on the right-hand side. For Problem 1, for instance, $\mathcal{R}[\hat{u}] = \hat{u}'' + \pi^2 \sin(\pi x)$. The derivatives $\hat{u}'$ and $\hat{u}''$ are computed via PyTorch's reverse-mode automatic differentiation through the full expression in eq. (18), which is exact up to floating-point arithmetic and does not incur any discretisation error in the derivatives themselves. This is the sense in which PINNs are mesh-free: neither the residual nor its derivatives depend on a discretisation of space.

Why no analogue of the BVP convergence theorem exists. For FDM the error bound of section 3.3 follows from linearity and the invertibility of A_h: once the truncation error is controlled the global error is controlled too. For a PINN the approximation is nonlinear in θ and the error depends on the trajectory of a non-convex optimiser. Wang et al. [9] analyse the PINN loss via its neural tangent kernel and show that the residual and boundary loss terms are driven at very different convergence rates under gradient descent, so that one component can stall while the other continues to decrease. Rahaman et al. [5] show that standard MLPs trained with gradient descent exhibit a *spectral bias* toward low-frequency components, so that oscillatory features of the target solution are learned much more slowly than smooth ones. Wang et al. [8] make this diagnosis PINN-specific and propose random Fourier-feature input embeddings [7] as one remedy.

4.3 Architecture and Training

The architecture is fixed across all three problems and across all values of N_c. Table 3 summarises it.

Parameter	**Value**
Hidden layers	4
Neurons per layer	32
Activation	$\tanh$
Initialisation	Xavier uniform, seed 42
Trainable params	≈ 3200

Table 3: PINN architecture, held constant throughout the experiments.

We use $\tanh$ rather than ReLU because the PDE residual requires $\hat{u}''$ and ReLU is not twice differentiable.

Two-phase training. Following Raissi et al. [6] we train in two phases. Phase 1 is 10,000 Adam steps at learning rate 10^{-3} with cosine annealing, which brings the loss from its random initialisation down to roughly 10^{-3}. Phase 2 is a single `torch.optim.LBFGS` call with strong Wolfe line search and up to 500 iterations, which polishes the solution the rest of the way. In our runs the L-BFGS phase is essential: without it the PINN error is roughly an order of magnitude worse at every N_c. L-BFGS is a quasi-Newton

method that approximates the Hessian from the last few gradients, and it plays the same role here as Newton FD plays in section 3.4.2 (quadratic-ish convergence once the iterate is close to a minimum).

Reproducibility. All experiments use a fixed seed (42) for NumPy, PyTorch, and weight initialisation. The PINN collocation points are deterministic (uniform interior grid at each N_c). This makes the runs deterministic modulo floating-point nondeterminism in CUDA; we run everything on CPU.

4.4 PINN Numerical Results

We train one PINN per problem per value of N_c, for $N_c \in \{10, 25, 50, 100, 250, 500, 1000\}$, and record the final L^∞ error, the final training loss, and the wall-clock time. Table 4 collects the accuracy numbers.

N_c	**Problem 1**	**Problem 2**	**Problem 3**
10	8.92×10^{-5}	1.76×10^{-5}	5.90×10^{-5}
25	9.72×10^{-6}	1.87×10^{-5}	6.13×10^{-5}
50	7.39×10^{-6}	2.03×10^{-5}	6.55×10^{-5}
100	6.44×10^{-6}	2.17×10^{-5}	6.79×10^{-5}
250	6.79×10^{-6}	2.15×10^{-5}	6.92×10^{-5}
500	6.91×10^{-6}	2.23×10^{-5}	6.98×10^{-5}
1000	6.44×10^{-6}	2.24×10^{-5}	7.07×10^{-5}

Table 4: PINN L^∞ error versus number of collocation points. After an initial decrease for the smallest N_c (Problem 1), the error is effectively constant in N_c on all three problems.

The effective convergence rate between successive N_c values is zero for every problem beyond the smallest grids: increasing N_c by an order of magnitude does not reduce the error. On Problem 2 and Problem 3 the error drifts slightly upward as N_c grows, consistent with the failure mode of Wang et al. [9]. The plateau is problem-dependent but consistent across N_c: roughly 6×10^{-6} for Problem 1, 2×10^{-5} for Problem 2, and 7×10^{-5} for Problem 3.

Precision is not the cause. An obvious candidate mechanism for the plateau is that 32-bit floating-point accumulation in the PINN residual prevents further progress. To rule this out, we re-ran the entire PINN sweep in double precision (`torch.set_default_dtype(torch.float64)`), keeping every other hyperparameter identical. The plateau drops by at most a factor of two (e.g. Problem 1 plateau moves from $\sim 6.4 \times 10^{-6}$ to $\sim 3.1 \times 10^{-6}$, and Problem 2 is essentially unchanged at $\sim 3 \times 10^{-5}$), which is far smaller than the four orders of magnitude of headroom that double precision provides. We conclude that float32 arithmetic is not load-bearing on these problems and that the plateau originates in the optimiser and the network's representational dynamics, exactly as predicted by the references above.

5 Comparison

The two methods are now compared on the four axes of section 1.

5.1 Accuracy versus Sampling Density

Figure 2 overlays the FDM and PINN accuracy sweeps on a single log-log axis per problem. For FDM the horizontal axis is the number of interior grid points n; for PINN it is the collocation count N_c. As discussed

in section 4.1 these axes measure different things, but both are the natural notion of *sampling density* for the respective method, and plotting them together makes the qualitative difference immediate.

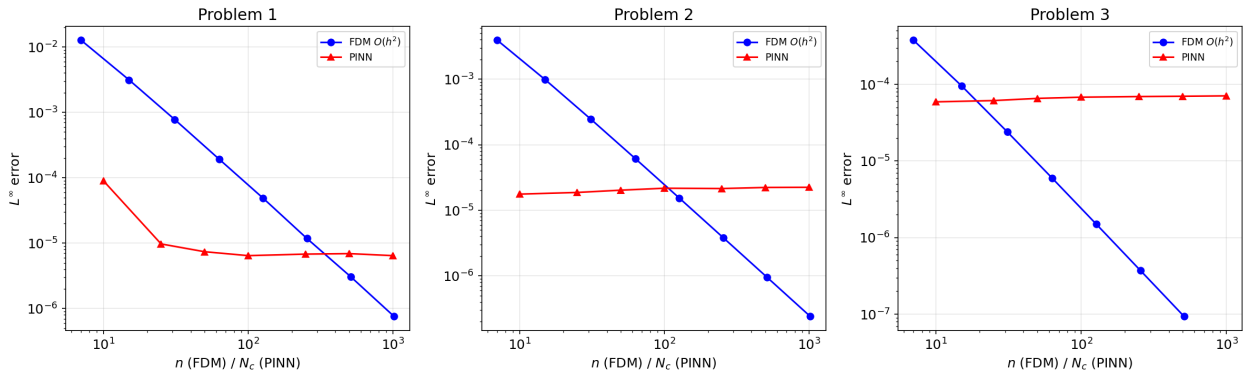

Figure 2: FDM (blue) and PINN (red) L^∞ error versus sampling density (grid points n for FDM, collocation points N_c for PINN). FDM exhibits the theoretical slope -2 on all three problems and reaches sub-10^{-6} accuracy by $n = 1023$. The PINN error plateaus around 6×10^{-6} on Problem 1 and around 2 to 7×10^{-5} on Problems 2 and 3, regardless of N_c.

FDM exhibits the straight-line slope -2 in log-log coordinates predicted by section 3.3 and achieved numerically in table 1. PINN plateaus essentially instantly: after the first doubling of N_c the error is effectively flat. The two methods cross at modest sampling density: on Problem 3, for example, FDM at $n = 63$ already gives 5.97×10^{-6}, beating the PINN plateau by an order of magnitude, and FDM at $n = 511$ outperforms the PINN by nearly three orders of magnitude.

5.2 Conditioning versus Loss Landscape

Both methods have an accuracy ceiling, but the two ceilings have very different character. Figure 3 presents the two side by side for Problem 1.

The FDM ceiling is analytic. From eq. (7) we know $\kappa(A_h)$ exactly, and the empirical curve matches it; the threshold at which linear-algebra roundoff overwhelms discretisation accuracy can be written down before running any experiment. We do not reach this threshold in our sweep: $\kappa(A_h)$ at $n = 1023$ is $\sim 4 \times 10^5$, so the worst-case amplification of double-precision roundoff is of order 10^{-10}, well below the 7.55×10^{-7} discretisation error at that grid.

The PINN ceiling is empirical. The training loss descends cleanly to $\sim 10^{-7}$, but the test error plateaus at $\sim 10^{-5}$ to 10^{-6}: loss and error decouple. There is no theoretical prediction for where this plateau sits; it must be measured after the fact.

5.3 Iterative Solver Convergence Rates

Table 5 puts all solvers used in the project on a single page. The course hierarchy is exactly as predicted: linear-convergence methods (bisection and Picard) need an order of magnitude more iterations than quadratic-convergence methods (Newton shooting and Newton FD). For comparison, the PINN is a quasi-Newton method (L-BFGS) on a non-convex loss and requires on the order of 10^4 iterations to reach its plateau.

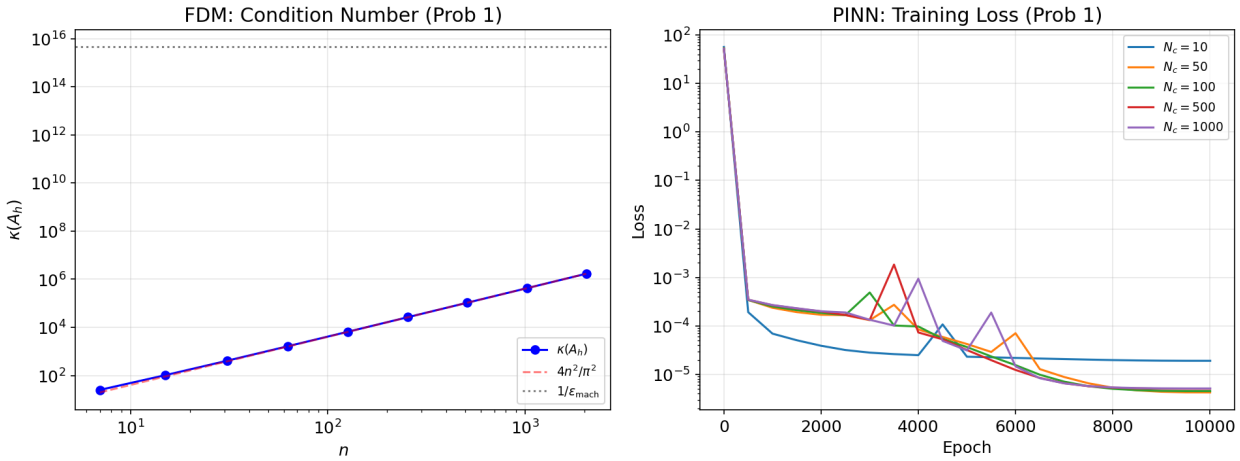

Figure 3: Two failure modes, same problem. Left: condition number $\kappa(A_h)$ versus n for the Problem 1 FDM matrix. The empirical curve (blue circles) matches the analytic $4(n+1)^2/\pi^2$ (red dashed). The grey horizontal line is $1/\epsilon_{\text{mach}}$ in double precision: FDM accuracy degrades once $\kappa \cdot \epsilon_{\text{mach}}$ approaches 1. Right: PINN training loss versus epoch on Problem 1 for several values of N_c, showing a complicated but consistent descent to $\sim 10^{-7}$ regardless of N_c.

Solver	**Asymptotic rate**	**Iterations to 10^{-12} (Prob 3)**
Bisection shooting	linear	41
Newton shooting	quadratic	5
Picard FD	linear	13
Newton FD	quadratic	2
PINN (Adam + L-BFGS)	non-convex	$\sim 10^4$ (plateau only)

Table 5: Summary of iterative solvers used in the project. The first four are the classical course methods, applied to Problem 3. The last row is the PINN optimiser, included for perspective.

5.4 Computational Cost

The wall-clock costs of the two methods differ by roughly five orders of magnitude. Table 6 reports representative numbers.

At the accuracy the PINN achieves ($\sim 10^{-5}$), FDM delivers the same accuracy at $n \leq 63$ in a few hundred microseconds, making the training-cost ratio roughly 10^5 against the PINN. PINN inference (400 microseconds, once trained) is essentially equal to the FDM solve time; the mesh-free evaluation does not help a 1D problem where the mesh is trivial to build.

6 Conclusions

Across all three test problems and on every axis we measured, the classical finite difference method is strictly better than the PINN for the class of problems under consideration:

- **Accuracy** is higher for FDM at every sampling density we tested beyond the smallest grids, with a clean second-order scaling that takes FDM below 10^{-6} at $n = 1023$. The PINN plateau sits at $\sim 10^{-5}$ on Problems 2 and 3 independent of N_c.

Method	Wall-clock time
FDM, Problem 1, $n = 63$	$\approx 210\mu s$
FDM, Problem 1, $n = 255$	$\approx 200\mu s$
FDM, Problem 1, $n = 1023$	$\approx 370\mu s$
FDM, Problem 3, $n = 255$	$\approx 1.2ms$
PINN, Problem 1, $N_c = 100$, training	$\approx 16s$
PINN, Problem 1, $N_c = 1000$, training	$\approx 74s$
PINN inference (any problem)	$\approx 400\mu s$

Table 6: Representative wall-clock times on CPU. FDM includes the full tridiagonal solve; PINN training includes 10k Adam steps and the L-BFGS refinement; PINN inference is a forward pass on 1000 test points from an already-trained model.

- **The failure mode is theoretically characterised** for FDM (condition-number-limited roundoff, reachable only at very fine grids), whereas the PINN plateau is empirical and not predictable from first principles; we verified that it is not a precision artefact.
- **Iterative convergence** on the nonlinear problem follows the course hierarchy exactly: quadratic methods (Newton shooting, Newton FD) dominate linear ones (bisection, Picard), and both are orders of magnitude faster to converge than the PINN optimiser.
- **Wall-clock cost** is roughly five orders of magnitude lower for FDM than for PINN training at the accuracy level the PINN actually achieves.

PINNs are not a drop-in replacement for classical numerical methods on problems where the classical methods already have complete theory. The settings where PINNs may be preferable are those where FDM is awkward or impossible: inverse problems, where Raissi et al. [6] demonstrate that PINNs can jointly recover the solution and unknown PDE parameters from sparse noisy data, and problems on irregular or high-dimensional domains where mesh generation becomes the bottleneck.

7 Further Research

Within the scope of the comparison itself, several extensions were planned but not executed in time for this submission. Each is documented here so that the gaps in the experimental story are explicit, and to record the state of the partial implementations that exist in the project repository.

Random Fourier-feature input embedding. Of the candidate mechanisms for the PINN accuracy plateau, the experiment in section 4.4 rules out floating-point precision and leaves spectral bias and the non-convex optimisation landscape as the remaining candidates. The cleanest test of the spectral-bias hypothesis is to prepend a random Fourier-feature input embedding [7, 8] to the network. The embedding

$$\gamma(x) = \big[\sin(2\pi Bx),\ \cos(2\pi Bx)\big], \quad B_i \sim \mathcal{N}(0, \sigma^2),$$

is sampled once at initialisation and held fixed; the network is then trained on $\gamma(x)$ rather than on x directly. If spectral bias is load-bearing, the Fourier-embedded PINN should reach a smaller plateau on Problem 1 (whose exact solution $\sin(\pi x)$ has obvious frequency content) than the baseline does. If the plateau is unchanged across problems, spectral bias is also ruled out and the remaining mechanism is non-convexity

of the loss landscape. The notebook `code/main_fourier.ipynb` contains the full implementation, a two-stage sweep (a σ pilot at $N_c = 100$ followed by a full N_c sweep at the per-problem winning σ), and the analysis plots; only the actual training runs were not completed in time. Other architectural and training-recipe modifications (loss reweighting, longer L-BFGS phases, adaptive collocation sampling) could be tested in the same framework.

Higher-order FDM via Richardson extrapolation. On the classical side, the natural follow-up is to increase the order of the FDM accuracy via Richardson extrapolation. The extrapolated solution $u_{\text{rich}} = (4\,u_{h/2} - u_h)/3$ cancels the leading $\mathcal{O}(h^2)$ error term and yields $\mathcal{O}(h^4)$ accuracy with no additional stencil work, which would push FDM well past its current advantage and reach the condition-number ceiling at much smaller n. This is a one-line change on top of the existing FDM solver and would extend table 1 with two more columns; it was deferred because the existing comparison already establishes the qualitative result.

Higher dimensions and irregular geometries. The most plausible setting in which PINNs could outperform FDM is in dimensions where mesh generation becomes expensive or the curse of dimensionality makes a regular grid impractical. A natural next experiment would be the same comparison on the 2D Poisson equation on an L-shaped domain, where the corner singularity and the non-rectangular boundary make classical methods awkward and where mesh-free residual sampling is at its most attractive. Conversely, extending FDM to 2D would require either a five-point Laplacian on a Cartesian grid (clean but inflexible) or a finite element discretisation, which is out of scope for this course.

Inverse problems. The clearest practical advantage of PINNs documented in the original Raissi et al. [6] paper is the inverse setting: given sparse noisy observations of a solution, recover both the solution and unknown parameters of the governing equation. There is no analogue of this for FDM, which solves only forward problems with known coefficients. A test problem would be Problem 2 with the coefficient $a(x) = 1 + x^2$ replaced by an unknown $a(x)$ to be recovered jointly with u from a handful of interior measurements. This is a qualitatively different experiment from the comparison done here and would require a separate paper.

References

[1] Gene H. Golub and James M. Ortega. *Scientific Computing and Differential Equations*. Academic Press, 1992.

[2] Boualem Khouider. MATH 449/549 chapter 4: Boundary value problems (slides). University of Victoria, Spring 2026, 2026.

[3] Boualem Khouider. MATH 449/549 lecture notes. University of Victoria, Spring 2026, 2026.

[4] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, 2007.

[5] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML 2019)*, pages 5301–5310, 2019.

[6] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[7] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*, 2020.

[8] Sifan Wang, Hanwen Wang, and Paris Perdikaris. On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 384:113938, 2021. doi: 10.1016/j.cma.2021.113938.

[9] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.